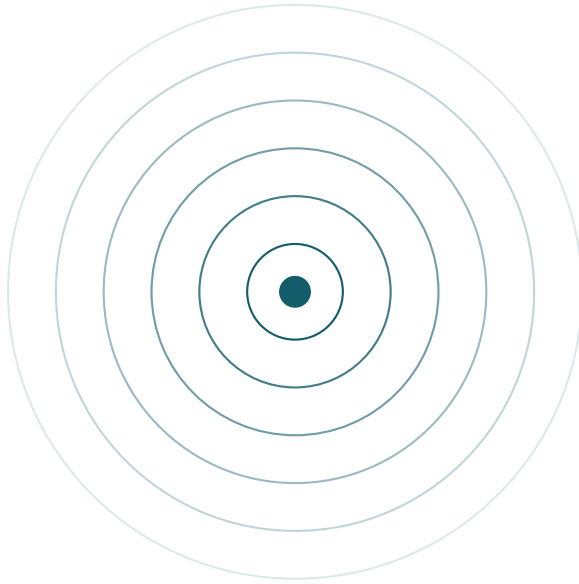


From bytes on a disk to counting words in Python



Text Files from Zero

A Sample Book

FIRST EDITION

Text Files from Zero

*A Sample Book That Exercises
Every Feature of the Series Template*

FIRST EDITION

Text Files from Zero: A Sample Book

Copyright © 2026 The Author. All rights reserved.

Revision History for the First Edition

2026-09-26 First release

The programs in this book are included for their instructional value. They have been tested with Python 3.14 on macOS, but no warranty, express or implied, is offered with respect to them, and no liability is accepted for any damages arising from their use. Third-party services and tools described here change over time; check their current documentation.

The program in Chapter 2 is short enough to type; every command and its output were run for this book.

Typeset in Crimson Pro, Source Sans 3, and Source Code Pro with XeLaTeX. Source: Markdown, converted with pandoc.

Contents

Preface	v
Who This Book Is For	v
How This Book Is Organized	v
How to Use This Book	v
What You Need Installed.	v
Conventions Used in This Book.	vi
 Part I. Understand	 1
 1. What a Text File Is	 2
Bytes First	2
Lines Are a Convention Too.	3
Lab: See the Bytes for Yourself.	3
Summary, Key Terms, and Review Questions	4
 Part II. Use	 5
 2. Counting Words with Python	 6
Reading a File the Right Way.	6
Running It.	7
When It Fails.	7
Lab: Count the Words in Your Own File	7
Summary, Key Terms, and Review Questions	8
 Appendices	 9
 A. Command Cheat Sheet	 9
Files and Bytes	9
Python.	9
 B. Glossary	 10

Index 11

Preface

This sample book exists to exercise every feature of the series template in a few pages. It explains, briefly and truthfully, what a text file is and how to count the words in one with Python. Read it next to its Markdown source to see which convention produces which result.

Who This Book Is For

You can open a terminal and type a command. You do **not** need to know anything about files, bytes, or Python beyond that. Everything else is explained when it's needed.

How This Book Is Organized

- **Part I** builds the mental model: what is actually inside a text file.
- **Part II** puts it to use: a ten-line Python program that counts words, and a lab.
- **Appendix A** is a cheat sheet; **Appendix B** is the glossary.

How to Use This Book

Read the chapters in order; they're short. **Type the code yourself** rather than copying it. Every chapter ends with a lab and a **Summary, Key Terms, and Review Questions** section.

What You Need Installed

- **Python 3.12 or newer** (check with `python3 --version`)
- **A terminal**: Terminal on macOS, any terminal on Linux, WSL on Windows

Conventions Used in This Book

- **Bold** marks a new term the first time it is explained. Every such term is also in Appendix B.
- Constant width marks code, commands, file names, and anything else you would type exactly.
- Code blocks show Python, shell commands, or program output. Shell commands are shown without a prompt, so you can paste them as they are.
- Three kinds of callout appear in the text:



A suggestion that saves time or trouble.



A general remark, or something to be aware of.



A mistake that can cost you data, money, or security. Read these twice.

Understand



What to ignore for now: file systems and inodes, the history of ASCII and code pages, and Unicode normalisation. None of it stops you from reading and writing text files correctly.

What a Text File Is

Before you can process a text file, you need a clear picture of what one is. This chapter gives you that picture: bytes on a disk, an encoding that turns them into characters, and the new-line that turns characters into lines.

The problem. I open `notes.txt` in an editor and see three short lines. A hex viewer shows the same file as a row of numbers.

The question. Which one is the file, and why does a program that counts lines sometimes disagree with my eyes?

Bytes First

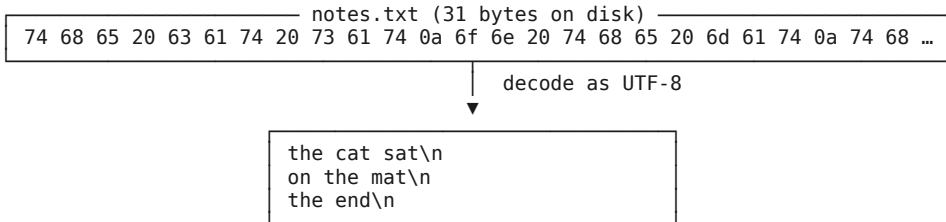
A **file** is a named sequence of **bytes** stored on a disk. A byte is a number from 0 to 255; nothing more. The file system remembers the name and the length, and it doesn't care what the numbers mean.

A **text file** is a file whose bytes are meant to be read as characters. The rule that maps bytes to characters is the file's **encoding**. Today that is almost always **UTF-8**, which uses one byte for plain English letters and two to four bytes for everything else.



“Plain text” is not a format of its own. It's an agreement between whoever wrote the bytes and whoever reads them: *use UTF-8*. When the two sides disagree, you see garbage such as `cafÃ©` instead of `café`.

Here is the whole idea in one picture. The same three lines, as the disk stores them and as an editor shows them:



Lines Are a Convention Too

A **line** is the text between two **newline** characters. The newline is an ordinary byte (0a, written \n in code). Editors hide it and start a new row instead; to a program it's just another character.



When a line count looks one short, check whether the file ends with a newline. `wc -l` counts newline characters, not rows, so a last line without one isn't counted.

Different systems historically chose different line endings. You'll meet all three:

System	Line ending	Bytes	Where you still see it
Unix, Linux, macOS	LF	0a	Almost every file on a server, every Git repository by default
Windows	CR LF	0d 0a	Files created by Notepad and older Windows tools; some CSV exports from spreadsheets
Classic Mac OS (before 2001)	CR	0d	Very old files only; treat it as a curiosity



Never "fix" line endings on a file you don't own by saving it in another editor. A shell script with CR LF endings fails on Linux with a confusing `bad interpreter` error, and a changed file can silently break a checksum or a signature.

Lab: See the Bytes for Yourself

`xxd` prints any file's bytes in hexadecimal, next to their characters.

1. Run `xxd notes.txt`. **Expected:**

```
00000000: 7468 6520 6361 7420 7361 740a 6f6e 2074 the cat sat.on t
00000010: 6865 206d 6174 0a74 6865 2065 6e64 0a he mat.the end.
```

- Count the `0a` bytes. **Expected:** three. Each `0a` is a newline, which `xxd` prints as a dot on the right, so the file has three lines.

Summary, Key Terms, and Review Questions

Summary

- A file is a named sequence of bytes; the disk stores numbers, not letters.
- An **encoding** maps bytes to characters. Use **UTF-8** unless you have a reason not to.
- A line ends at a **newline** byte. Line endings differ between systems.

Key Terms

Term	Meaning
File	A named sequence of bytes stored on a disk
Encoding	The rule that turns bytes into characters, such as UTF-8
Newline	The character (<code>\n</code> , byte <code>0a</code>) that ends a line

Review Questions

- Why can a text editor and a hex viewer show the “same” file so differently?
- What does `wc -l` actually count?
- What goes wrong when a writer and a reader disagree about the encoding?

You understand this when you can explain why `café` takes 4 characters but 5 bytes in UTF-8.

PART II

Use



What to ignore for now: every option of every tool; the few you need are shown.

Counting Words with Python

Now that you know a text file is bytes plus an encoding, you can read one correctly from Python. This chapter builds a ten-line program that counts words, runs it from the shell, and shows what its errors mean.

The problem. I want to know which words appear most in a file, and counting by hand is hopeless.

The question. How do I read a text file correctly from Python and count what's in it?

Reading a File the Right Way

Python's `pathlib` module represents a file path as an object. Its `read_text()` method reads the bytes and decodes them in one step. Always pass the encoding; the default depends on the operating system.

```
"""Count how often each word appears in a text file."""
import sys
from collections import Counter
from pathlib import Path

def count_words(path: Path) -> Counter:
    text = path.read_text(encoding="utf-8")
    return Counter(text.lower().split()) # split() breaks on any whitespace

if __name__ == "__main__":
    counts = count_words(Path(sys.argv[1]))
    for word, n in counts.most_common(3):
        print(f"{word:<6} {n}")
```

A **Counter** is a dictionary that counts things: give it a list, and it maps each item to how many times it appeared. `most_common(3)` returns the three largest counts.

Running It

Save the program as `count_words.py` next to `notes.txt`, then run:

```
python3 count_words.py notes.txt
```

```
the    3
cat    1
sat    1
```

For a quick total, a one-liner is enough. It's long, so the book wraps it; paste it as one line:

```
python3 -c "from pathlib import Path; print(sum(len(line.split()) for line in
↪ Path('notes.txt').read_text(encoding='utf-8').splitlines()))"
```

8



`python3 -c` runs a string as a program. It's handy for checks like this one, but anything you'll run twice belongs in a file.

When It Fails

If the file doesn't exist, Python stops with a **traceback**: the chain of calls that led to the error, with the error itself on the last line. Read tracebacks from the bottom up.

```
FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'
```



Don't hide this error with a bare `except: .` A program that silently prints nothing when its input is missing looks like it worked, and its empty output can overwrite a good report.

Lab: Count the Words in Your Own File

Pick any text file you have, or create one with three lines of your own. Then:

1. Run `wc -w yourfile.txt` and note the number of words.
2. Run `python3 count_words.py yourfile.txt` and compare the top three words with what you expect.

3. Add a line with capital letters (The Cat) and run it again.

Expected result: the counts for the and cat go up by one each, because the program lowercases the text before counting. `wc -w` and the sum of all counts from the program should agree.

Summary, Key Terms, and Review Questions

Summary

- `Path.read_text(encoding="utf-8")` reads and decodes a file in one step.
- A **Counter** counts items; `most_common(n)` gives the top *n*.
- A **traceback** reads bottom-up: the last line is the error.

Key Terms

Term	Meaning
<code>pathlib</code>	The standard-library module for file paths
<code>Counter</code>	A dictionary subclass that counts items
<code>Traceback</code>	The report Python prints when an error stops a program

Review Questions

1. Why should you pass `encoding="utf-8"` to `read_text()`?
2. Why does the program call `lower()` before `split()`?
3. Which line of a traceback do you read first?

You understand this when you can predict the output of `count_words.py` for a file you wrote yourself, before you run it.

Command Cheat Sheet

The commands used in this book, in one place.

Files and Bytes

Command	What it does
<code>wc -l file</code>	Count newline characters (lines)
<code>wc -w file</code>	Count words (runs of non-whitespace)
<code>wc -c file</code>	Count bytes
<code>xxd file</code>	Show the bytes in hexadecimal, with their characters
<code>file notes.txt</code>	Guess what kind of data a file holds (ASCII text, UTF-8 Unicode text, ...)

Python

Snippet	What it does
<code>Path("f.txt").read_text(encoding="utf-8")</code>	Read and decode a whole file
<code>Counter(words).most_common(3)</code>	The three most frequent items
<code>python3 -c "..."</code>	Run a one-line program

Glossary

Term	Meaning
Byte	A number from 0 to 255; the unit a disk stores
Counter	A dictionary from Python's <code>collections</code> module that counts items
Encoding	The rule that maps bytes to characters, such as UTF-8
File	A named sequence of bytes stored on a disk
Line	The text between two newline characters
Newline	The character that ends a line: <code>\n</code> , byte <code>0a</code>
<code>pathlib</code>	Python's standard-library module for file paths
Text file	A file whose bytes are meant to be read as characters
Traceback	The report Python prints when an error stops a program
UTF-8	The dominant text encoding: one to four bytes per character

Index

B

bytes, 2

C

Counter (collections), 6

E

encoding, 2

F

file, 2

H

hex viewer, 2

L

line, 3

line ending, 3

CR LF, 3

LF, 3

N

newline, 3

P

pathlib, 6

T

text file, 2

traceback, 7

U

UTF-8, 2

Text Files from Zero

Every program reads files, and most of the bugs around them come from one missing idea: a text file is bytes plus an agreement about how to read them. This short sample book gives you that idea and a ten-line Python program to put it to work.

You'll learn how to:

- See the bytes inside any file, and why line endings differ
- Read a file correctly in Python and count its words
- Read a traceback from the bottom up